

Programming for AI

AI-2001

Inheritance

National University of Computer & Emerging Sciences – NUCES – Karachi



National University of Computer & Emerging Sciences – NUCES – Karachi

Course Code: AI-2001	Programming for AI
----------------------	--------------------

Contents

1. Inheritance	2
1.1 Syntax for Basic Inheritance	2
2. Types of Inheritance in Python	2
2.1 Single Inheritance	3
2.2 Multiple Inheritance	3
2.3 Multilevel Inheritance	4
2.4 Hierarchical Inheritance	5
2.5 Hybrid Inheritance	6

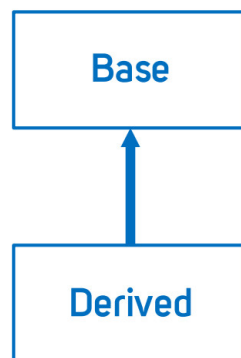
Talha Shahid

1. Inheritance

Inheritance is a key feature of the object-oriented programming model. It allows for code reuse by enabling the creation of a new class based on an existing one, eliminating the need to build it from scratch.

Through inheritance, a child class inherits the attributes and can access all data members and methods of the parent class. Additionally, the child class can override the parent class's methods with its unique implementations. This part of the tutorial will cover inheritance in depth.

In Python, a derived class can inherit from a base class by placing the base class name in parentheses after the derived class name.



1.1 Syntax for Basic Inheritance

In Python, inheritance is straightforward. We declare a child class by placing the parent class name in parentheses after the child class name.

```
class ParentClass:
    # Parent class implementation

class ChildClass(ParentClass):
    # Child class implementation, which inherits from ParentClass
```

2. Types of Inheritance in Python

Python supports several types of inheritance:

1. **Single Inheritance**
2. **Multiple Inheritance**
3. **Multilevel Inheritance**
4. **Hierarchical Inheritance**
5. **Hybrid Inheritance**

2.1 Single Inheritance

Single inheritance is when a single child class inherits from a single parent class.

Example: Creating an **Employee** class that inherits from **Person**.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

class Employee(Person):
    def __init__(self, name, age, position):
        super().__init__(name, age)
        self.position = position

    def display_employee(self):
        print(f"Position: {self.position}")

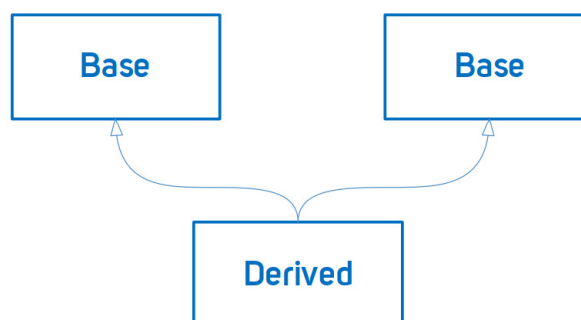
# Usage
emp = Employee("Muhammad Salman", 19, "Manager")
emp.display() # Inherited from Person
emp.display_employee() # Specific to Employee
```

Output

Name: Muhammad Salman, Age: 19
Position: Manager

Explanation: **Employee** inherits from **Person**, so it can access **Person's display()** method, and it also has its specific method **display_employee()**.

2.2 Multiple Inheritance



Multiple inheritance is when a child class inherits from more than one parent class.

Example: Creating a **Manager** class that inherits from both **Person** and **Employee**.

```
class Company:
    def __init__(self, company_name):
        self.company_name = company_name
```

```

def show_company(self):
    print(f"Company: {self.company_name}")

class Person:
    def __init__(self, name):
        self.name = name

    def show_name(self):
        print(f"Name: {self.name}")

class Manager(Person):
    def __init__(self, name, company_name):
        Person.__init__(self, name)
        Company.__init__(self, company_name)

# Usage
mgr = Manager("Safi ur Rehman", "System Limited")
mgr.show_name()
mgr.show_company()

```

Output

```

Name: Safi ur Rehman
Company: System Limited

```

Explanation: Manager inherits attributes and methods from both Company and Person classes.

2.3 Multilevel Inheritance



In multilevel inheritance, a class derives from another derived class, creating a chain of inheritance.

Example: Creating a chain where **Person** → **Employee** → **Manager**.

```

class Person:
    def __init__(self, name):
        self.name = name

    def show_name(self):
        print(f"Name: {self.name}")

class Employee(Person):
    def __init__(self, name, emp_id):
        super().__init__(name)
        self.emp_id = emp_id

    def show_employee(self):
        print(f"Employee ID: {self.emp_id}")

```

```

class Manager(Employee):
    def __init__(self, name, emp_id, department):
        super().__init__(name, emp_id)
        self.department = department

    def show_manager(self):
        print(f"Department: {self.department}")

# Usage
mgr = Manager("Dev Kumar", "E123", "Artificial Intelligence")
mgr.show_name()
mgr.show_employee()
mgr.show_manager()

```

Output

```

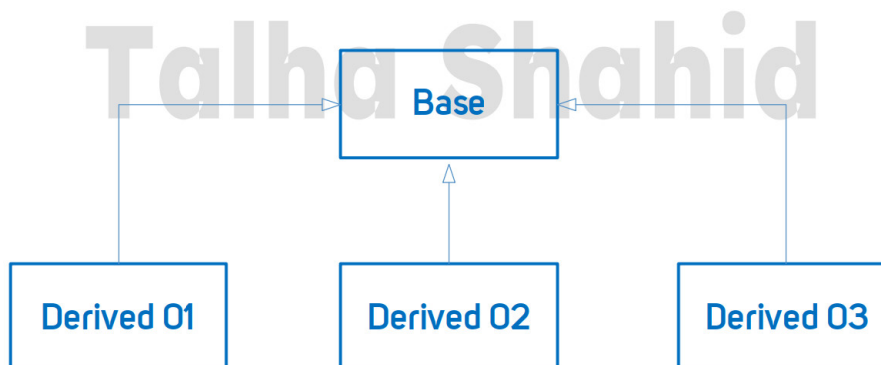
Name: Dev Kumar
Employee ID: E123
Department: Artificial Intelligence

```

Explanation: **Manager** inherits from **Employee**, which in turn inherits from **Person**, forming a multilevel inheritance hierarchy.

2.4 Hierarchical Inheritance

In hierarchical inheritance, multiple child classes inherit from a single parent class.



Example: **Student** and **Teacher** classes both inherit from **Person**.

```

class Person:
    def __init__(self, name):
        self.name = name

    def show_name(self):
        print(f"Name: {self.name}")

class Student(Person):
    def __init__(self, name, student_id):
        super().__init__(name)
        self.student_id = student_id

    def show_student(self):
        print(f"Student ID: {self.student_id}")

```



```

class Teacher(Person):
    def __init__(self, name, subject):
        super().__init__(name)
        self.subject = subject

    def show_teacher(self):
        print(f"Subject: {self.subject}")

# Usage
student = Student("Muhammad Umair Gul", "24K-0000")
teacher = Teacher("Talha Shahid", "Programming for AI")
student.show_name()
student.show_student()
teacher.show_name()
teacher.show_teacher()

```

Output

```

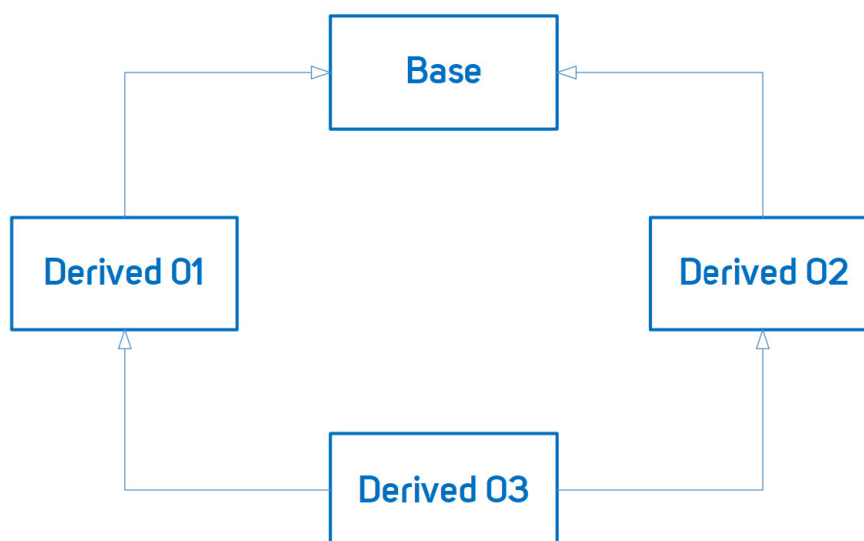
Student ID: 24K-0000
Name: Muhammad Umair Gul
Subject: Programming for AI

```

Explanation: Both **Student** and **Teacher** inherit from **Person**, which allows them to share **Person's** `show_name()` method.

2.5 Hybrid Inheritance

Hybrid inheritance combines two or more types of inheritance. This often requires careful planning to avoid the **diamond problem**—an issue in languages that support multiple inheritance where an ambiguity arises about which class to inherit from if a class appears in multiple places in the hierarchy.



Example: Hybrid inheritance structure with **Person**, **Employee**, and **Manager**.

```

class Person:

```

```

def __init__(self, name):
    self.name = name

def show_name(self):
    print(f"Name: {self.name}")

class Employee(Person):
    def __init__(self, name, emp_id):
        super().__init__(name)
        self.emp_id = emp_id

    def show_employee(self):
        print(f"Employee ID: {self.emp_id}")

class PartTime:
    def __init__(self, hours):
        self.hours = hours

    def show_hours(self):
        print(f"Hours: {self.hours}")

class PartTimeEmployee(Employee, PartTime):
    def __init__(self, name, emp_id, hours):
        Employee.__init__(self, name, emp_id)
        PartTime.__init__(self, hours)

# Usage
part_time_emp = PartTimeEmployee("Areeba Imran", "PT123", 20)
part_time_emp.show_name()
part_time_emp.show_employee()
part_time_emp.show_hours()

```

Output

```

Name: Areeba Imran
Employee ID: PT123
Hours: 20

```

Explanation: `PartTimeEmployee` inherits from both `Employee` (which itself inherits from `Person`) and `PartTime`, combining multiple types of inheritance.